

API Gateway Walkthrough

Full Technical Documentation

Project: Fideon OS Platform
Backend: FastAPI + Uvicorn + Gunicorn
Auth: Supabase JWT Tokens
Date: 2026-03-16

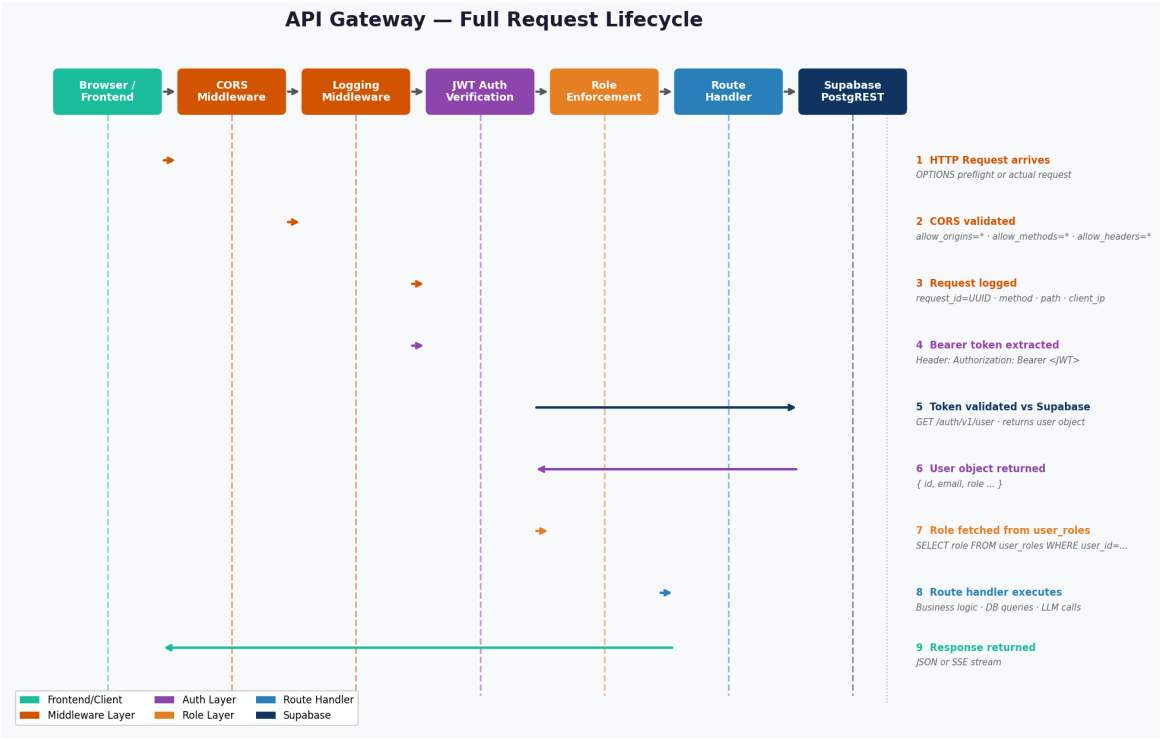


Figure 1 — Full Request Lifecycle Overview

1. Architecture Overview

The Fideon OS backend is a **FastAPI** application served by **Gunicorn with 4 Uvicorn workers** on port 8080. It acts as the central API gateway for all platform operations — chat, device management, pod activation, federated learning, and admin control.

Technology Stack

Component	Technology	Details
HTTP Server	Gunicorn + Uvicorn	4 workers, port 8080, UvicornWorker class
Framework	FastAPI	Python 3.11, async/await throughout
Auth Provider	Supabase Auth	JWT tokens validated server-side
Database	Supabase PostgreSQL	Accessed via PostgREST API + supabase-py
LLM Routing	litellm	Unified interface for all LLM providers
Logging	structlog	JSON-formatted, PII-scrubbed
Container	Docker	Non-root user (appuser:1001)
CORS	CORSMiddleware	allow_origins=[*] (permissive)

2. Middleware Chain

Every HTTP request passes through a **two-layer middleware chain** before reaching any route handler. CORS runs first (FastAPI's built-in), followed by the custom RequestLoggingMiddleware. Auth and role checks happen inside individual route handlers as dependency-injected functions.

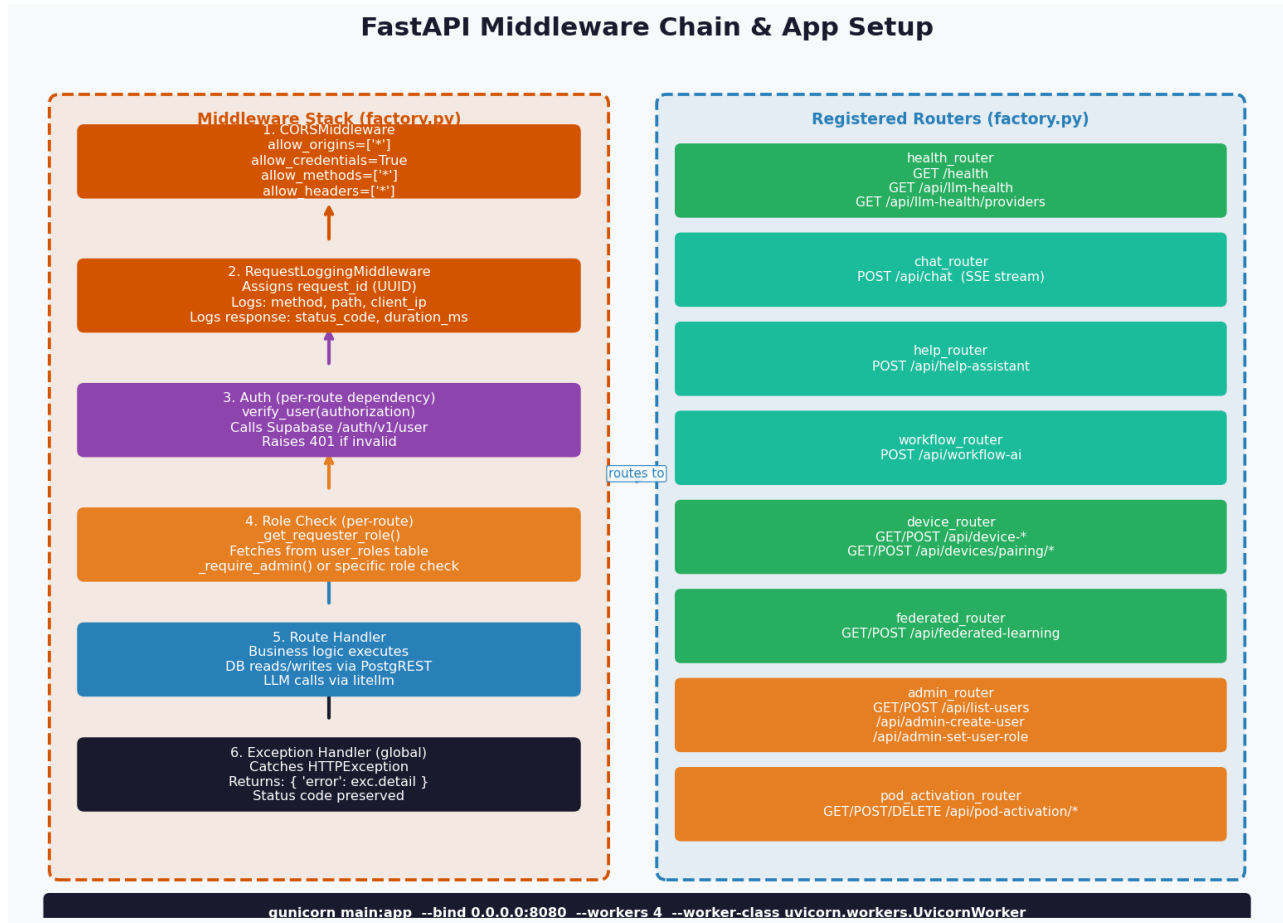


Figure 2 — Middleware Chain & Router Registration

Middleware Execution Order

Order	Middleware	Layer	Responsibility
1	CORSMiddleware	Framework built-in	Validates Origin, sets CORS headers, handles OPTIONS preflight
2	RequestLoggingMiddleware	Custom (logger/)	Assigns request_id UUID, logs method+path+IP, logs status+duration
3	verify_user() (per-route)	Route dependency	Extracts Bearer token, validates against Supabase /auth/v1/user
4	_get_requester_role()	Route helper	Queries user_roles table for caller's role string
5	_require_admin() / role check	Route logic	Raises 403 if role insufficient for the endpoint
6	Route Handler	Business logic	Executes actual logic: DB reads, LLM calls, response
7	http_exception_handler	Global exception	Catches HTTPException, returns {"error": detail}

3. JWT Authentication Flow

Authentication uses **Supabase JWT tokens**. The frontend obtains a JWT after login via the Supabase Auth SDK, and passes it in the Authorization: Bearer <token> header. The backend validates the token by calling Supabase's /auth/v1/user endpoint — it does **not** verify the JWT signature locally.

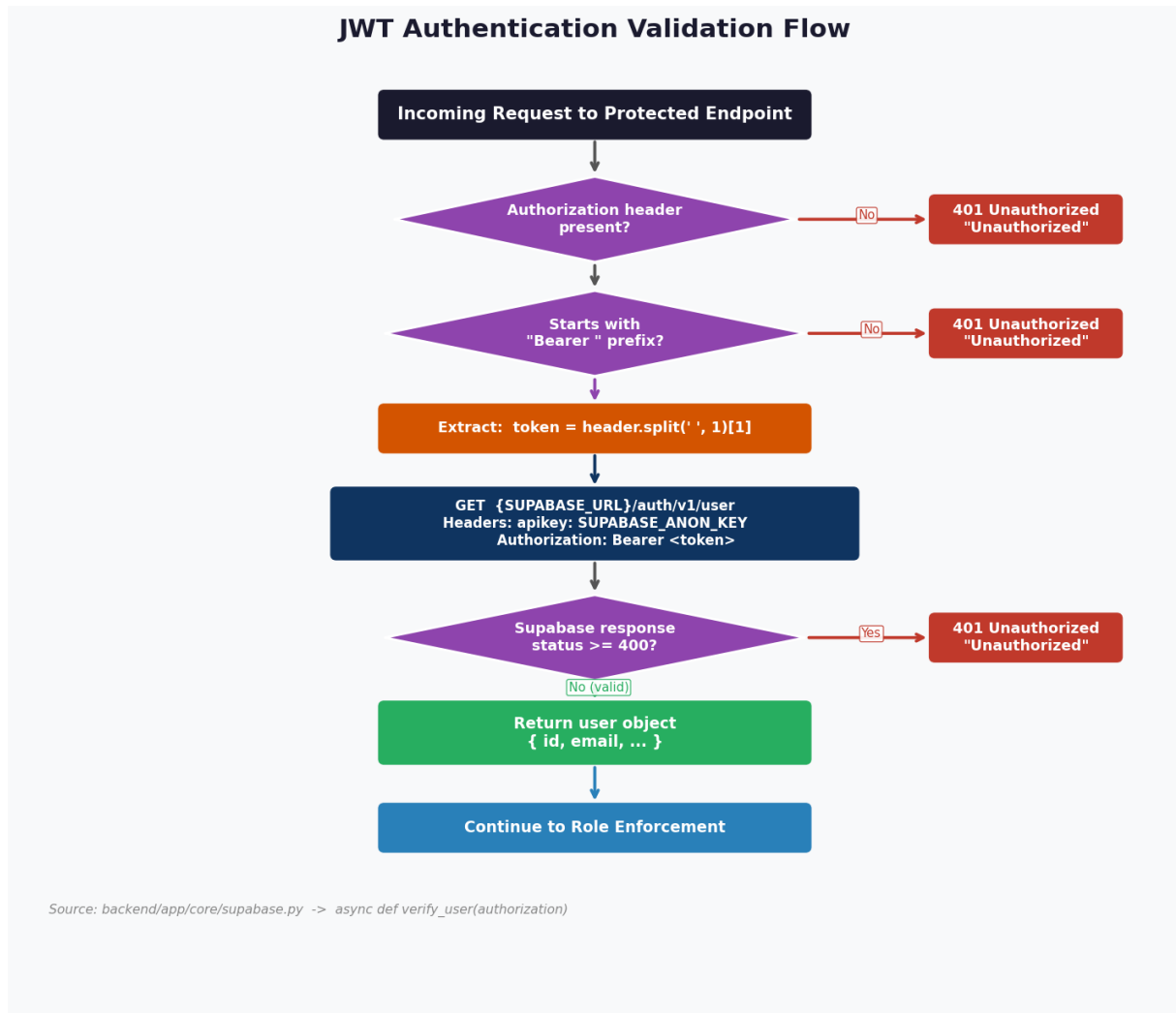


Figure 3 — JWT Token Validation Flow

verify_user() — Source Code (core/supabase.py)

```
async def verify_user(authorization: Optional[str]) -> Dict[str, Any]:
    if not authorization or not authorization.lower().startswith('bearer '):
        raise HTTPException(status_code=401, detail='Unauthorized')
    token = authorization.split(' ', 1)[1]
    async with httpx.AsyncClient(timeout=30) as client:
        resp = await client.get(
            f'{SUPABASE_URL}/auth/v1/user',
            headers={
                'apikey': SUPABASE_ANON_KEY,
                'Authorization': f'Bearer {token}',
            },
        )
    if resp.status_code >= 400:
        raise HTTPException(status_code=401, detail='Unauthorized')
    return resp.json()
```

4. Role Enforcement

After JWT validation, the backend fetches the caller's role from the `user_roles` Supabase table. Role enforcement is applied **per-endpoint** — different routes require different minimum roles.

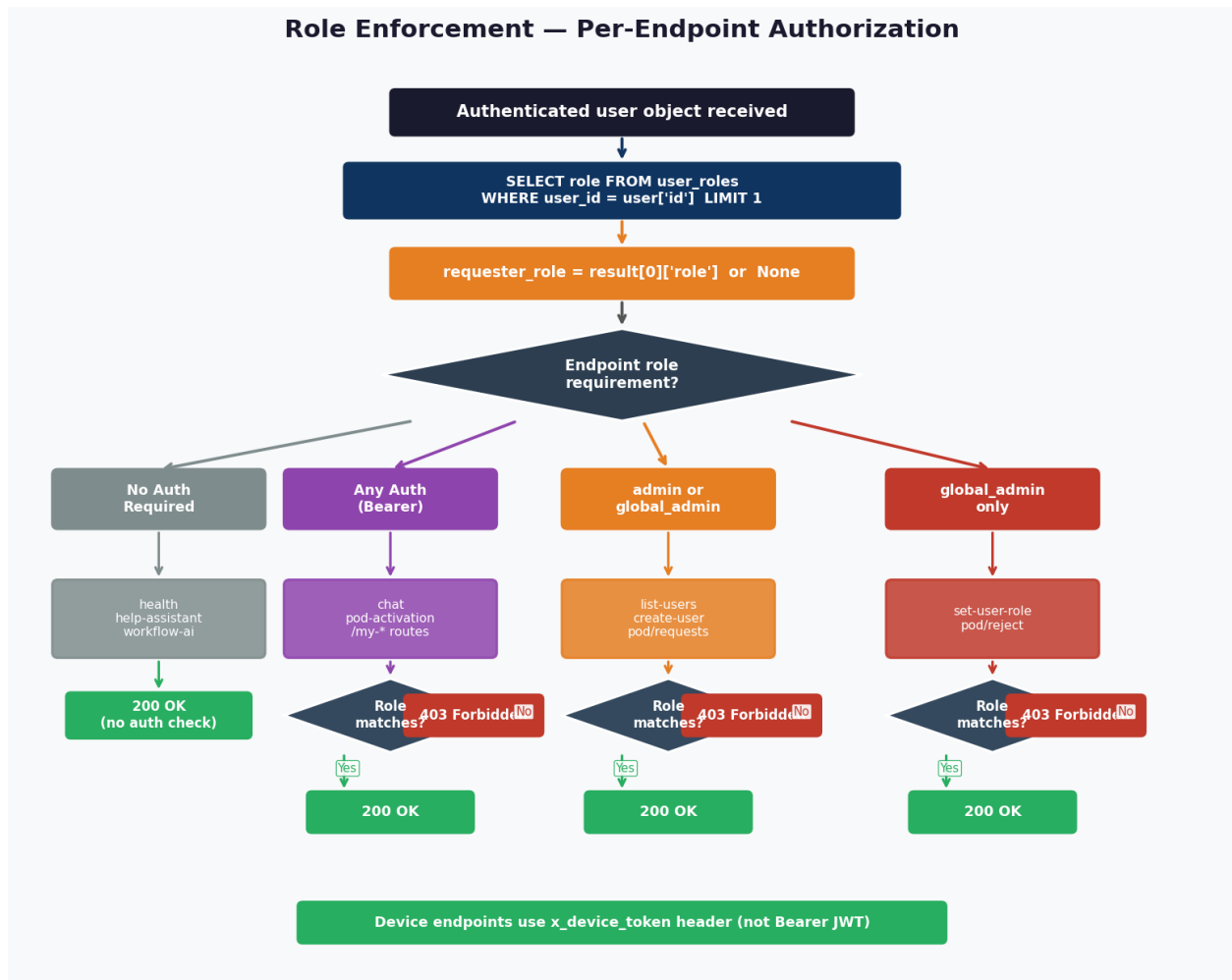


Figure 4 — Per-Endpoint Role Enforcement Flow

Role Check Source Code (routes/admin.py, routes/pod_activation.py)

```
async def _get_requester_role(authorization):
    requester = await verify_user(authorization)
    requester_roles = await postgrest_get('user_roles',
        f'select=role&user_id=eq.{requester["id"]}&limit=1')
    requester_role = requester_roles[0].get('role') if requester_roles else None
    return requester, requester_role

def _require_admin(role: Optional[str]) -> None:
    if role not in {'admin', 'global_admin'}:
        raise HTTPException(status_code=403, detail='Admin access required')
```

5. Complete API Endpoint Map

The platform exposes **25 API endpoints** across 8 route modules. Each endpoint has a specific auth requirement: no auth, Bearer JWT, or device token — plus a minimum role for protected routes.

Complete API Endpoint Map — Auth Requirements					
	Method	Path	Auth	Min Role	Description
HEALTH	GET	/health	None	-	Basic health check
	GET	/api/llm-health	None	-	LLM provider status
	GET	/api/llm-health/providers	None	-	Detailed provider probes
DEVELOPER / CHAT	POST	/api/chat	Bearer	Any auth	Stream chat (SSE)
	POST	/api/help-assistant	None	-	Help assistant stream
	POST	/api/workflow-ai	None	-	Workflow parse / assist
	GET	/api/device-models	Device	x_device	List device models
	POST	/api/device-checkin	Device	x_device	Device check-in
	POST	/api/device-register	None	-	Register device
	POST	/api/devices/pairing/start	Bearer	Any auth	Initiate device pairing
	GET	/api/devices/pairing/status/{id}	Bearer	Any auth	Get pairing status
	POST	/api/devices/pairing/confirm	None	-	Confirm device pairing
	GET/POST	/api/federated-learning	Device	x_device	Federated learning ops
POD / ACTIVATION	GET	/api/pod-activation/my-activations	Bearer	Any auth	My activated pods
	GET	/api/pod-activation/my-requests	Bearer	Any auth	My pod requests
	POST	/api/pod-activation/request	Bearer	Any auth	Create activation request
	GET	/api/pod-activation/requests	Bearer	admin+	All requests (admin)
	POST	/api/pod-activation/{id}/approve	Bearer	admin+	Approve request
	POST	/api/pod-activation/{id}/reject	Bearer	global_admin	Reject request
	GET	/api/pod-activation/user/{id}/activations	Bearer	admin+	User activations
	POST	/api/pod-activation/allocate	Bearer	admin+	Allocate model to user
	DELETE	/api/pod-activation/allocations/{id}	Bearer	admin+	Deallocate model
	ADMIN	GET	/api/list-users	Bearer	admin+
POST		/api/admin-create-user	Bearer	admin+	Create or update user
POST		/api/admin-set-user-role	Bearer	global_admin	Set user role

Auth & Role Legend

No Auth

Device Token

admin / global_admin

Bearer JWT

Any authenticated

global_admin only

Figure 5 — All 25 Endpoints with Auth Requirements

Endpoint Summary by Module

Module	Router	Endpoints	Auth
Health	health_router	3	None (public)
Chat	chat_router	1	Bearer JWT (any auth)
Help	help_router	1	None (public)
Workflow	workflow_router	1	None (public)
Device	device_router	6	Bearer JWT or x_device_token

Federated	federated_router	1	x_device_token
Pod Activation	pod_activation_router	9	Bearer JWT (role varies)
Admin	admin_router	3	Bearer JWT (admin+)

6. LLM Provider Fallback Chain

The /api/chat endpoint uses **litellm** to route requests through a waterfall of LLM providers. If the primary provider fails (timeout, rate limit, error), the next provider is tried automatically. All responses are streamed back to the client as **Server-Sent Events (SSE)**.

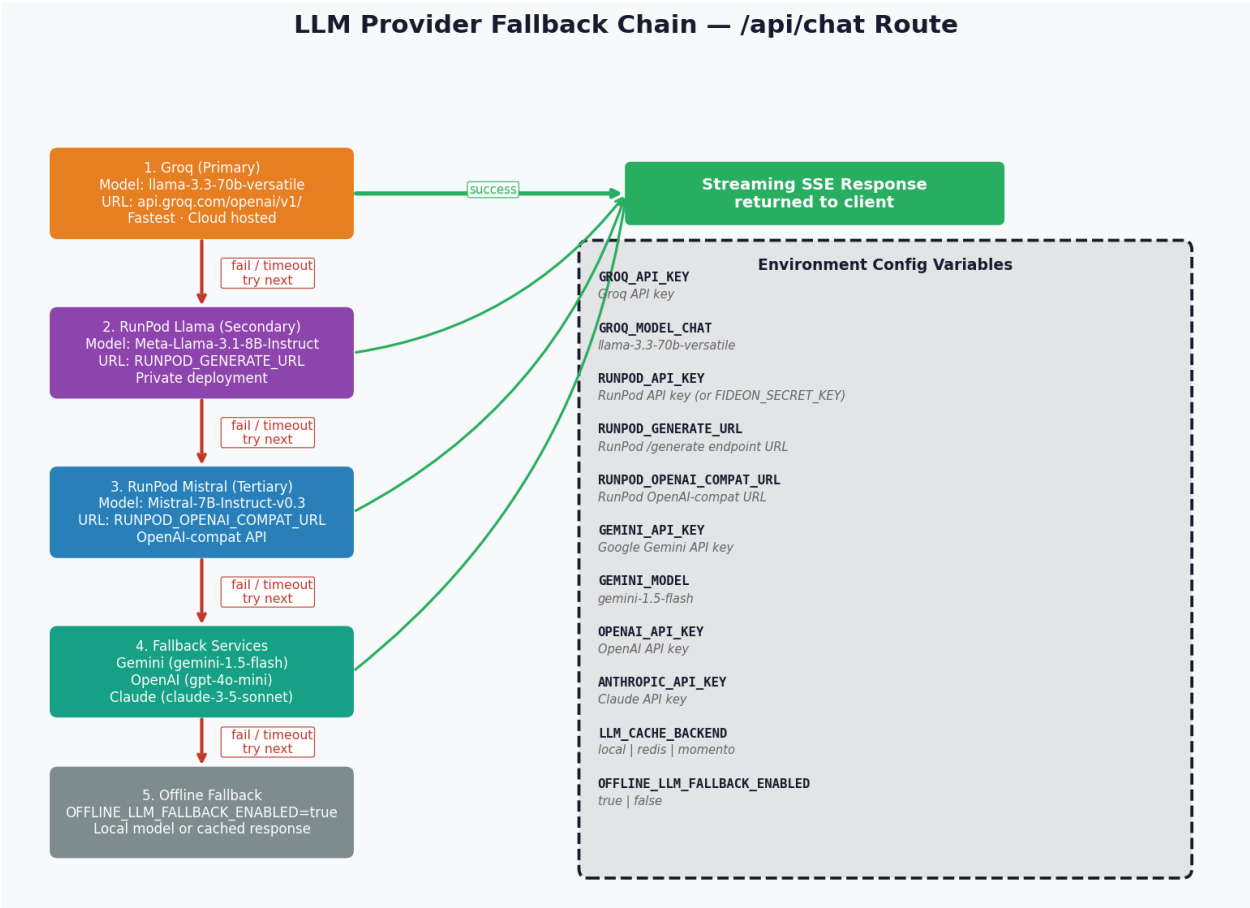


Figure 6 — LLM Provider Fallback Chain

Provider Priority Order

Priority	Provider	Model	URL / Service
1 (Primary)	Groq	llama-3.3-70b-versatile	api.groq.com/openai/v1/
2	RunPod Llama	Meta-Llama-3.1-8B-Instruct	RUNPOD_GENERATE_URL
3	RunPod Mistral	Mistral-7B-Instruct-v0.3	RUNPOD_OPENAI_COMPAT_URL
4a	Gemini	gemini-1.5-flash	generativelanguage.googleapis.com
4b	OpenAI	gpt-4o-mini	api.openai.com/v1/
4c	Claude	claude-3-5-sonnet-20241022	api.anthropic.com/v1/messages
5 (Offline)	Fallback Svc	(local/cached)	OFFLINE_LLM_FALLBACK_ENABLED=true

7. Device Authentication & Pairing

Physical devices (AI edge boxes) use a separate authentication mechanism: **device tokens** passed via the `x_device_token` header. Devices are paired to user accounts via a QR-code / URL-based pairing flow.

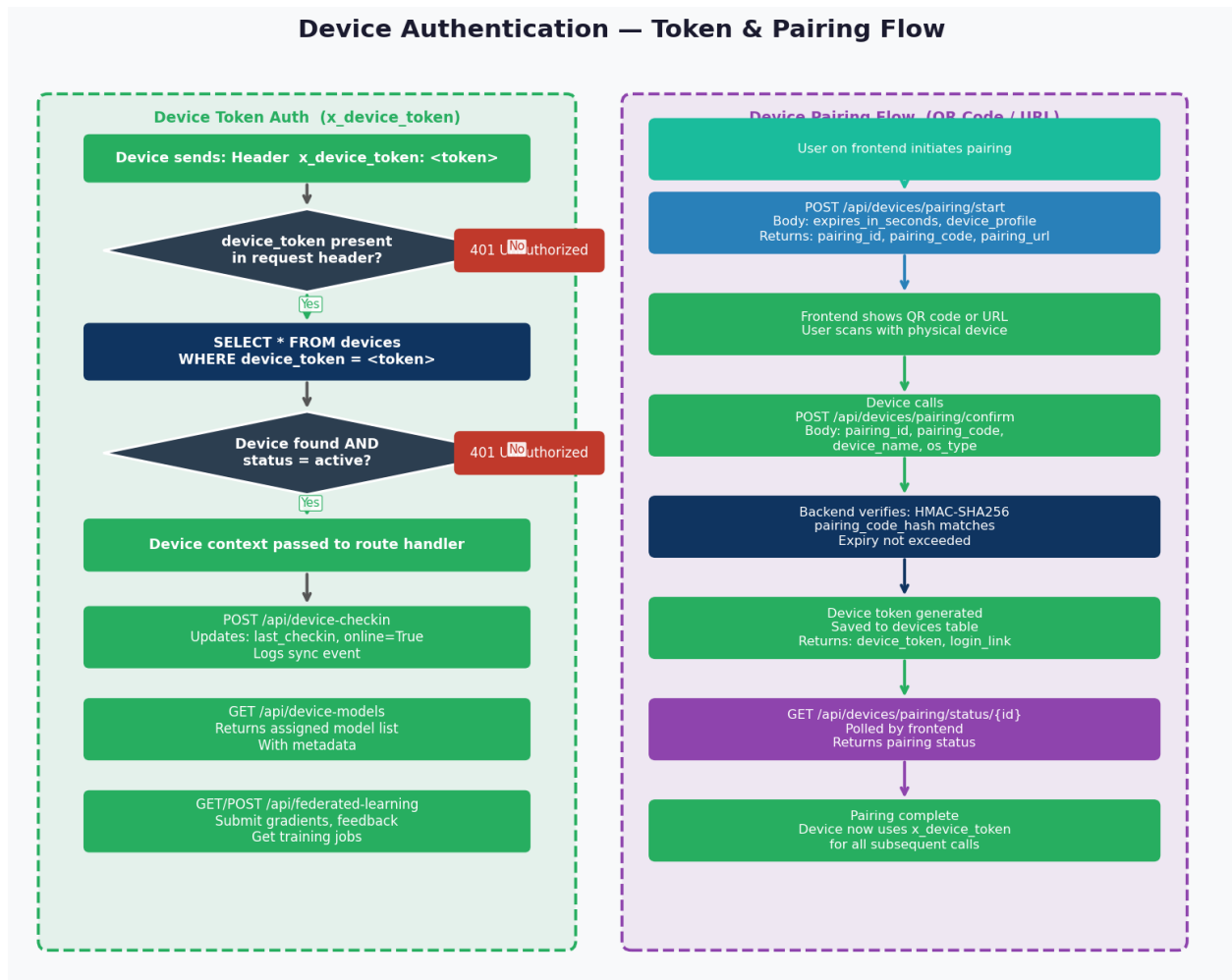


Figure 7 — Device Token & Pairing Flow

Device Token vs JWT Token

Property	Bearer JWT Token	<code>x_device_token</code>
Used by	Browser / Frontend (users)	Physical edge devices
Header	Authorization: Bearer <token>	<code>x_device_token: <token></code>
Issued by	Supabase Auth (login)	Backend (pairing confirm)
Validated via	GET <code>/auth/v1/user</code> (Supabase)	DB lookup in devices table
Endpoints	chat, pod-activation, admin, pairing	device-models, checkin, federated
Expiry	Supabase session TTL	Stored in DB, revocable

8. Error Response Handling

All errors are handled by a **global exception handler** registered in `factory.py`. Every error returns a consistent JSON body. Sensitive fields are never included in error messages — they are scrubbed from all log output by the PII scrubber.

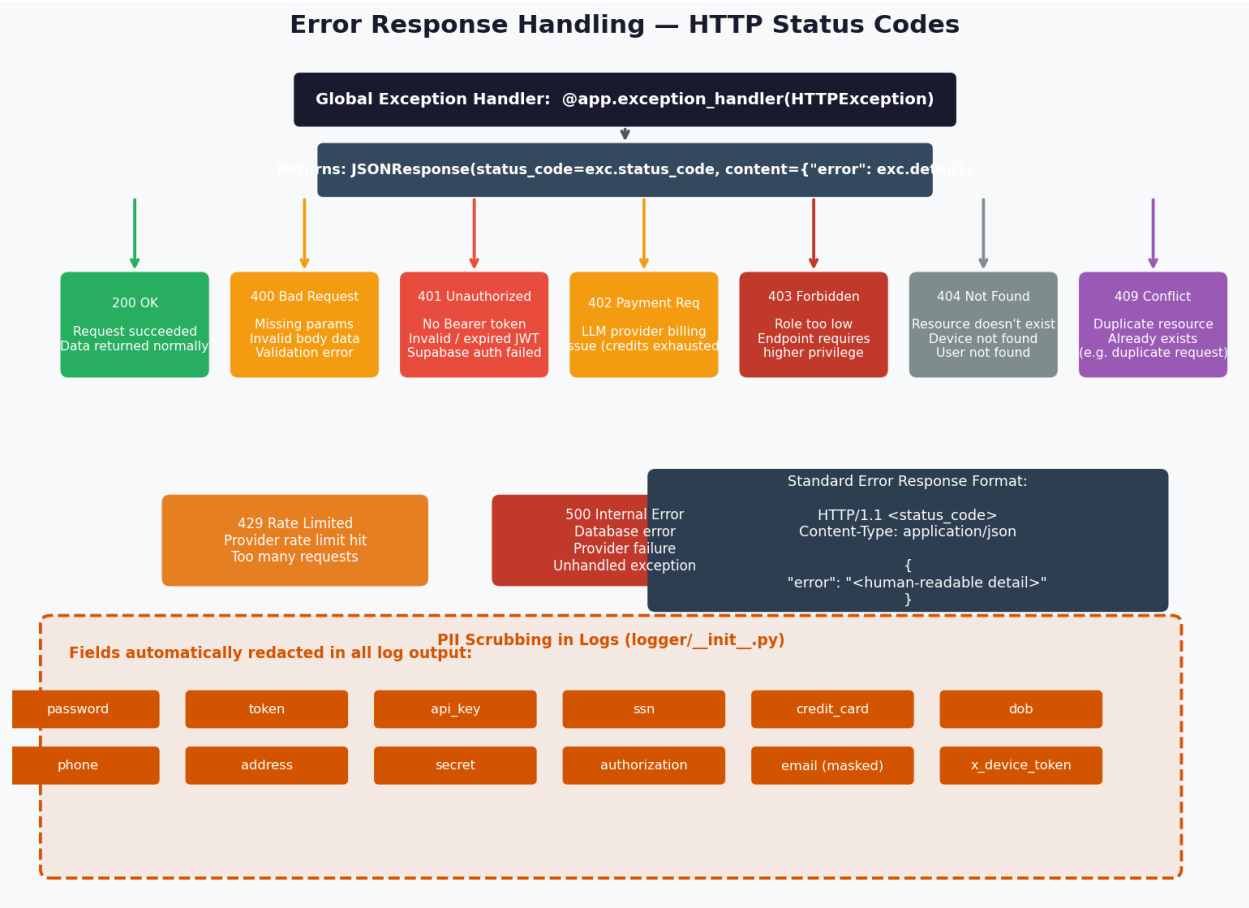


Figure 8 — Error Handling & Status Code Reference

HTTP Status Code Reference

Status	Meaning	When It Occurs
200	OK	Request succeeded normally
400	Bad Request	Missing/invalid parameters, validation error
401	Unauthorized	No Bearer token, invalid JWT, Supabase auth failed
402	Payment Required	LLM provider billing issue (credits exhausted)
403	Forbidden	User role is too low for this endpoint
404	Not Found	Resource not found (device, user, request)
409	Conflict	Duplicate resource (e.g. duplicate pod request)
429	Rate Limited	LLM provider rate limit hit
500	Internal Error	Database failure, provider error, unhandled exception

9. Security Configuration

This section summarises the security posture of the API gateway — what protections are in place and which gaps exist.

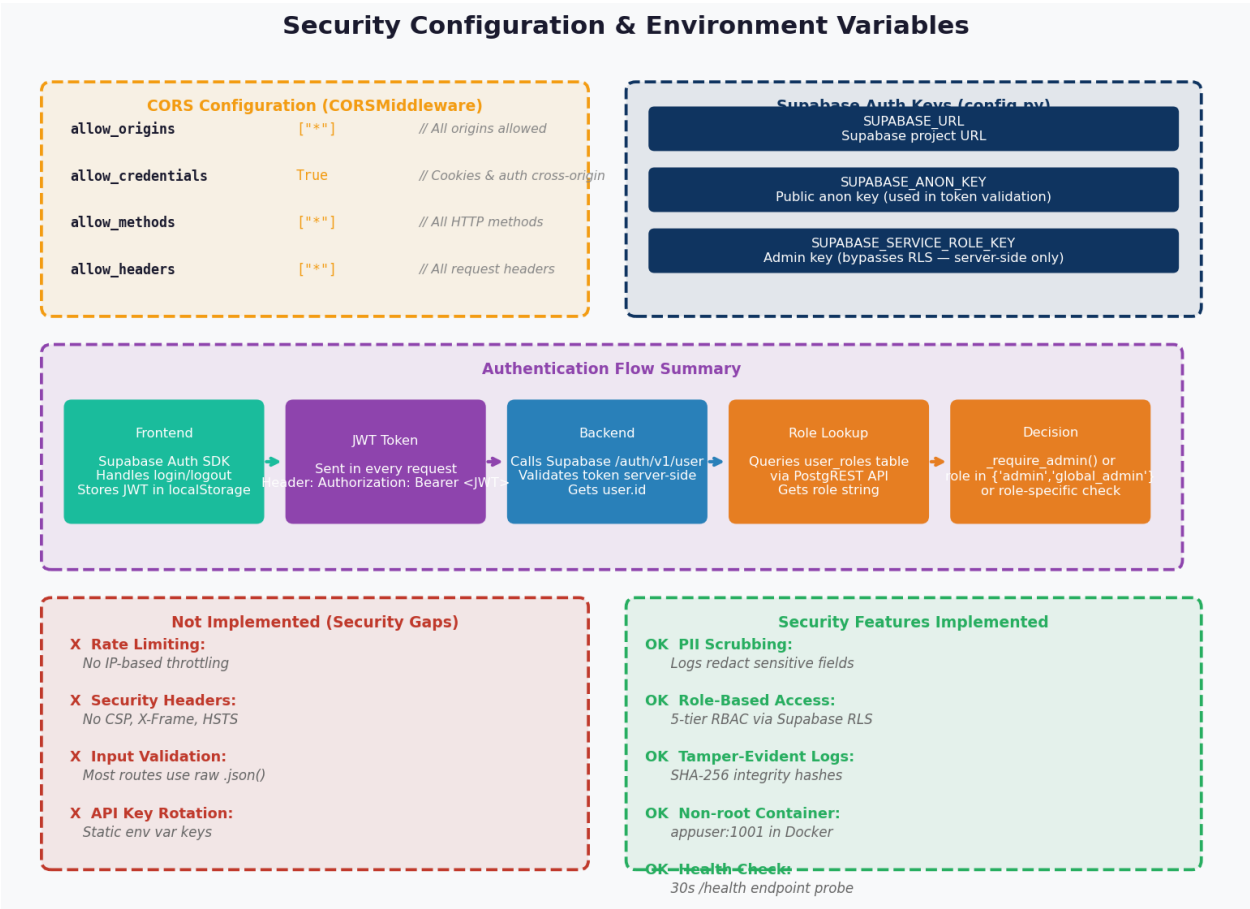


Figure 9 — Security Configuration Overview

Environment Variables Reference

Variable	Purpose	Required
SUPABASE_URL	Supabase project endpoint	Yes
SUPABASE_ANON_KEY	Token validation key	Yes
SUPABASE_SERVICE_ROLE_KEY	Admin DB operations	Yes
GROQ_API_KEY	Primary LLM provider	Yes
RUNPOD_API_KEY / FIDEON_SECRET_KEY	RunPod access	Optional
RUNPOD_GENERATE_URL	RunPod /generate endpoint	Optional
GEMINI_API_KEY	Gemini fallback	Optional
OPENAI_API_KEY	OpenAI fallback	Optional
ANTHROPIC_API_KEY	Claude fallback	Optional

LLM_CACHE_BACKEND	local redis momento	Optional
OFFLINE_LLM_FALLBACK_ENABLED	Enable offline fallback	Optional